

Accelerated Opinion Evolution for the Friedkin-Johnsen Model

Anonymous Author

Abstract

Online social networks have become a central medium for large-scale opinion sharing and formation, shaping opinion dynamics across domains such as politics, public health, and misinformation diffusion. In this context, the Friedkin–Johnsen (FJ) model is a widely used model for studying opinion formation in social networks, capturing the effects of internal opinions and social influence. However, existing models typically account only for initial opinions and neighbor interactions, while neglecting latest opinion updates and historical memory, despite their prevalence in real-world social networks. To address this limitation, we propose an opinion evolution framework that extends the classical FJ model and instantiate it through three iterative algorithms, capturing latest opinion updates, self-interaction effects, and historical influence. We theoretically prove that the proposed algorithms converge to the same equilibrium opinion, while achieving faster convergence rates than existing methods. Extensive experiments on a large collection of real-world networks demonstrate that our methods are efficient and effective, and remain scalable to networks with millions of nodes.

Keywords

Opinion Dynamics, Online Social Networks, Friedkin–Johnsen Model, Iterative Algorithms

ACM Reference Format:

Anonymous Author. 2026. Accelerated Opinion Evolution for the Friedkin-Johnsen Model. In . ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Social media and online social networks have fundamentally changed how opinions are formed, exchanged, and disseminated at scale. Understanding opinion dynamics in such networks is critical for analyzing key social phenomena, including polarization, disagreement, and conflict. To this end, a variety of mathematical models have been proposed, among which the FriedkinJohnsen (FJ) model has emerged as one of the most widely adopted frameworks due to its simplicity and strong explanatory power.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

A central object of interest in the FJ model is the equilibrium opinion vector, which characterizes individuals expressed opinions at steady state and serves as the basis for measuring polarization and disagreement. Existing studies have extensively analyzed the properties, interpretations, and applications of the FJ model. However, a key limitation shared by most existing formulations is that opinion updates are restricted to pairwise interactions among nearest neighbors. In contrast, real-world social interactions often extend beyond immediate neighbors: individuals may be influenced by second-order or higher-order connections through long-range ties, indirect information exposure, or social bridges. Such higher-order interactions have been shown to play a fundamental role in information diffusion and opinion formation, yet they are not explicitly captured by the classical FJ framework. However, a key limitation shared by most existing formulations of the FJ model is that opinion updates are driven solely by individuals initial opinions and static neighbor interactions. In real-world social networks, opinion formation is often a dynamic process in which individuals react to the most recently expressed opinions, exhibit self-reinforcement effects, and are influenced by their own historical opinions over time. Ignoring such latest updates and historical memory may limit the models ability to fully capture realistic opinion evolution processes observed in large-scale online platforms.

To address this limitation, we develop an opinion evolution framework that extends the classical FJ model to account for temporal dependencies in opinion updates, including the influence of recently expressed opinions and historical self-interaction. This framework gives rise to a family of iterative update rules, which preserve the equilibrium structure of the original FJ model while enabling faster convergence in practice.

While a variety of iterative acceleration and optimization techniques have been extensively studied for the DeGroot model, their extension to the FriedkinJohnsen (FJ) model remains largely unexplored. This gap is nontrivial: unlike DeGroot dynamics, the FJ model explicitly incorporates agents intrinsic opinions, leading to fundamentally different equilibrium structures and computational challenges. As a result, techniques that are effective for DeGroot-type consensus models cannot be directly applied to the FJ setting.

From a computational perspective, incorporating temporal dependencies such as latest opinion updates and historical memory leads to iterative opinion dynamics that differ from the standard FJ update rule. While these extensions preserve the same equilibrium opinion as the classical FJ model, they introduce additional dependencies across iterations, making efficient convergence a key computational challenge

for large-scale networks. To address this issue, we design efficient iterative algorithms that explicitly exploit the structure of the proposed update rules, and provide theoretical guarantees on convergence behavior.

Extensive experiments on real-world networks demonstrate that the proposed methods are both efficient and scalable, enabling the computation of equilibrium opinions on graphs with millions of nodes. The results show that incorporating latest opinion updates and historical influence significantly accelerates convergence in practice, while maintaining the equilibrium structure of the classical FJ model, making the proposed framework suitable for large-scale opinion analysis.

2 Related Work

3 Preliminaries

In this section, we introduce some useful notations and fundamental concepts of the FJ model of opinion formation for the convenience of description of problem formulation and algorithm.

3.1 Notations

We use $\mathbb{R}$ to represent the set of real numbers and regular lowercase letters like a, b, c to represent scalars in $\mathbb{R}$. Bold lower letters are column vectors, e.g., $\mathbf{a}, \mathbf{b}, \mathbf{c}$. Bold capital letters, e.g., $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are matrices. To denote specific elements in vectors and matrices, We denote A_{ij} as the entry of $\mathbf{A}$ at i -th row and j -th column and a_i as i -th element of vector $\mathbf{a}$. And we use $\mathbf{a}^\top$ and $\mathbf{A}^\top$ to denote transpose of vector $\mathbf{a}$ and matrix $\mathbf{A}$. We use $\mathbf{1}$ to denote the vector of appropriate dimensions with all entries being ones and use $\mathbf{e}_u$ to represent the u -th standard basis vector of appropriate dimensions. $\mathbf{I}$ denotes the identify matrix of appropriate dimensions.

3.2 Graph and Matrix Definitions

Let $\mathcal{G} = (V, E)$ be an unweighted simple graph with $n = |V|$ nodes and $m = |E|$ edges, where V is the set of nodes and $E \subseteq V \times V$ is the set of edges. The graph $\mathcal{G}$ is either a directed graph or an undirected graph depending on the context. The adjacency relation of all nodes in $\mathcal{G}$ is encoded in the adjacency matrix $\mathbf{A}$, whose entry $A_{ij} = 1$ if i and j are adjacent, and $A_{ij} = 0$ otherwise. For any given node i , N_i denote the set of its neighbors, meaning $N_i = \{j : (i, j) \in E\}$. The degree d_i of any node i is defined by $d_i = \sum_{j=1}^n A_{ij}$. The degree diagonal matrix of $\mathcal{G}$ is $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_n)$. The degree vector $\mathbf{d}$ is given by $\mathbf{d} = \mathbf{D}\mathbf{1}$. The transition matrix of an unbiased random walk on graph $\mathcal{G}$ is $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$, which is a row-stochastic matrix. The Laplacian matrix $\mathbf{L}$ of $\mathcal{G}$ is $\mathbf{L} = \mathbf{D} - \mathbf{A}$, which is a symmetric semi-positive definite matrix. Matrix $\mathbf{L}$ is singular, and thus it is irreversible. For a matrix $\mathbf{M}$, if the magnitudes of all the eigenvalues of $\mathbf{M}$ are less than 1, then the series $\sum_{k=0}^{\infty} \mathbf{M}^k$ is a Neumann series and converges to $(\mathbf{I} - \mathbf{M})^{-1}$.

3.3 Friedkin–Johnsen (FJ) Model

As one of the first opinion dynamics models, the FJ model is an extension of the DeGroot's opinion model. In the DeGroot model, every node has only one opinion that is updated as the weighted average of its neighbors. Different from the DeGroot model, in the FJ model, each node $i \in V$ has two different kinds of opinions: one is the internal (or innate) opinion s_i , the other is the expressed opinion z_i . The internal opinion s_i is assumed to remain constant, private to node i , while the expressed opinion z_i evolves as a weighted average of its corresponding internal opinion s_i and the expressed opinions of i 's neighbors.

Given a graph $G = (V, E)$, let $\mathbf{s}$ denote the internal opinion vector, with each entry satisfying $s_i \in [0, 1]$. The formulation of the equilibrium opinion expressed in the FJ model is an iterative form such that

$$z_i^{(t+1)} = \frac{s_i + \sum_{j \in N_i} A_{ij} z_j^{(t)}}{1 + \sum_{j \in N_i} A_{ij}}. \quad (1)$$

As in the FJ iterative formula above, the opinion of each node in the FJ model is updated as the weighted average of the neighbor's expressed opinion z_j and its own internal opinion s_i . When t approaches infinity, the updated expressed opinion was shown to converge to a unique equilibrium opinion $\mathbf{z}$. So the iterative update in Equation 1 can be written compactly as $\mathbf{z}^{(t+1)} = (\mathbf{I} + \mathbf{D})^{-1}(\mathbf{s} + \mathbf{A}\mathbf{z}^{(t)})$ in vector form, which expresses the FJ dynamics as a linear transformation applied at each iteration. Then the system of equilibrium expressed opinion vector $\mathbf{z}$ is equivalent to: $(\mathbf{I} + \mathbf{L})\mathbf{z} = \mathbf{s}$, where $\mathbf{I}$ is the identify matrix, and $\mathbf{L} = \mathbf{D} - \mathbf{A}$ is the Laplacian matrix. When $\mathcal{G}$ is connected and $A_{ij} \geq 0$, the matrix $\mathbf{I} + \mathbf{L}$ is symmetric positive definite, ensuring existence and uniqueness of $\mathbf{z}$. This characterization motivates estimators and algorithms that exploit the spectral and structural properties of the Laplacian.

3.4 Game-theoretic Interpretation

From a game-theoretic perspective, the classical FJ model admits an equivalent interpretation as the equilibrium of a quadratic utility game. Recall that the FJ equilibrium satisfies $(\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z} = \mathbf{s}$. For undirected graphs, the matrix $\mathbf{I} + \mathbf{D} - \mathbf{A}$ corresponds to a symmetric, positive definite Laplacian-based operator, which allows the equilibrium to be characterized as the unique minimizer of a strongly convex quadratic objective.

This formulation further admits a decentralized interpretation. Updating z_i according to the FJ rule is equivalent to each agent i minimizing the local quadratic cost

$$(z_i - s_i)^2 + \sum_{j \in N(i)} w_{i,j} (z_i - z_j)^2, \quad (2)$$

given the opinions of its neighbors. Thus, opinion averaging can be viewed as a sequence of best-response updates in a complete-information game, where each agent selects an opinion to balance conformity with its intrinsic belief. The resulting Nash equilibrium coincides with the minimizer of the global social cost and is unique due to strong convexity.

4 Existing Algorithm

5 Accelerated Opinion Iteration

We study accelerated iterative schemes for computing the equilibrium of the Friedkin-Johnsen (FJ) model. Beyond the classical synchronous update, we consider sequential and latest-opinion-based iterations that immediately exploit newly updated opinions. These schemes are further extended with self-interaction, yielding a family of accelerated algorithms that preserve the same equilibrium as the standard FJ model while achieving faster convergence. Their convergence properties are analyzed in the following subsections.

5.1 Sequential Latest-Opinion Updates

The classical FJ iteration updates all opinions synchronously, using only the opinion profile from the previous iteration. In contrast, we consider a sequential update scheme in which opinions are revised one by one, and newly updated opinions are immediately used in subsequent updates.

Under this scheme, opinions are updated in a coordinate-wise manner. Specifically, when updating the i -th opinion at iteration $t+1$, the most recent values of opinions $1, \dots, i-1$ are used whenever available, while the remaining opinions retain their values from iteration t :

$$z_i^{(t+1)} = \frac{s_i + \sum_{j=1}^{i-1} a_{ij} z_j^{(t+1)} + \sum_{j=i+1}^n a_{ij} z_j^{(t)}}{1 + \sum_{j \in N_i} a_{ij}}. \quad (3)$$

This update reduces to the classical synchronous FJ iteration when all opinions on the right-hand side are taken from iteration t . As shown later, the sequential latest-opinion update preserves the FJ equilibrium and typically converges faster due to the immediate utilization of newly updated opinions.

The sequential latest-opinion update admits a compact matrix representation. Specifically, it can be written as a linear fixed-point iteration $\mathbf{z}^{(k+1)} = \mathbf{B}\mathbf{z}^{(k)} + \mathbf{c}$, where $\mathbf{B} = (\mathbf{I} + \mathbf{D} - \mathbf{A}_{low})^{-1} \mathbf{A}_{up}$, $\mathbf{c} = (\mathbf{I} + \mathbf{D} - \mathbf{A}_{low})^{-1} \mathbf{s}$.

Algorithm 1 implements the proposed sequential latest-opinion iteration. At each iteration, the algorithm performs a single forward sweep over the nodes in a fixed order. When updating node i , the most recent opinion values are used whenever available, i.e., newly updated values from the current sweep are immediately incorporated, while the remaining opinions are taken from the previous iterate. This latest-available policy enables faster information propagation along the sweep and distinguishes the method from the classical synchronous FJ iteration.

The update combines the aggregated neighbor influence with the internal opinion s_i and applies a local normalization by $1 + d_i$. The iteration terminates when a prescribed tolerance is met or a maximum number of iterations is reached.

Let $\epsilon^{(k)} = \mathbf{z}^{(k)} - \mathbf{z}$ denote the error at iteration k , where $\mathbf{z}$ is the FJ equilibrium. We next establish the convergence of the sequential latest-opinion iteration.

The theorem below establishes the convergence of the latest-value updates.

THEOREM 5.1 (CONVERGENCE AND LINEAR RATE OF LATEST-OPINION ITERATION). *Consider the latest-opinion iteration*

$$\mathbf{z}^{(t+1)} = \mathbf{B}\mathbf{z}^{(t)} + \mathbf{c}, \quad (4)$$

where $\mathbf{B} = (\mathbf{I} + \mathbf{D} - \mathbf{A}_{low})^{-1} \mathbf{A}_{up}$. The iteration converges to the unique FJ equilibrium $\mathbf{z} = (\mathbf{I} + \mathbf{D} - \mathbf{A})^{-1} \mathbf{s}$ for any initialization $\mathbf{z}^{(0)}$. Moreover, the convergence is linear: for any induced matrix norm,

$$\|\mathbf{z}^{(t)} - \mathbf{z}\| \leq \|\mathbf{B}\|^t \|\mathbf{z}^{(0)} - \mathbf{z}\|.$$

And the error satisfies $\|\mathbf{z}^{(t)} - \mathbf{z}\|_v \leq \frac{q^t}{1-q} \|\mathbf{z}^{(1)} - \mathbf{z}^{(0)}\|_v$, which shows the convergence rate of the algorithm.

5.2 Proof of Theorem 5.1

We provide a concise proof of the convergence of the latest-value iteration. Recall that the iteration can be written as

$$\mathbf{z}^{(t+1)} = \mathbf{B}\mathbf{z}^{(t)} + \mathbf{c}, \quad (5)$$

where $\mathbf{B} = (\mathbf{I} + \mathbf{D} - \mathbf{A}_{low})^{-1} \mathbf{A}_{up}$ and $\mathbf{c} = (\mathbf{I} + \mathbf{D} - \mathbf{A}_{low})^{-1} \mathbf{s}$.

Let $\mathbf{z}$ denote the fixed point of (5), which satisfies

$$\mathbf{z} = \mathbf{B}\mathbf{z} + \mathbf{c}. \quad (6)$$

Define the error vector $\boldsymbol{\epsilon}^{(t)} := \mathbf{z}^{(t)} - \mathbf{z}$. Subtracting (6) from (5) yields the linear error recursion

$$\boldsymbol{\epsilon}^{(t+1)} = \mathbf{B}\boldsymbol{\epsilon}^{(t)}. \quad (7)$$

By induction, we obtain $\boldsymbol{\epsilon}^{(t)} = \mathbf{B}^t \boldsymbol{\epsilon}^{(0)}$.

If $\rho(\mathbf{B}) < 1$, then $\mathbf{B}^t \rightarrow \mathbf{0}$ as $t \rightarrow \infty$, and hence $\boldsymbol{\epsilon}^{(t)} \rightarrow \mathbf{0}$ for any initialization $\mathbf{z}^{(0)}$. Therefore, the latest-value iteration converges to the unique fixed point $\mathbf{z}$, which coincides with the equilibrium of the Friedkin-Johnsen model. Furthermore,

$$\|\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}\| = \|\mathbf{z} - \mathbf{z}^{(t)} - (\mathbf{z} - \mathbf{z}^{(t+1)})\|$$

$$\geq \|\mathbf{z} - \mathbf{z}^{(t)}\| - \|\mathbf{z} - \mathbf{z}^{(t+1)}\| \geq (1 - q) \|\mathbf{z} - \mathbf{z}^{(t)}\|,$$

on the other hand,

$$\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)} = \mathbf{B}(\mathbf{z}^{(t)} - \mathbf{z}^{(t-1)}),$$

and hence

$$\|\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}\|_v \leq q \|\mathbf{z}^{(t)} - \mathbf{z}^{(t-1)}\|_v.$$

As a result,

$$\|\mathbf{z} - \mathbf{z}^{(t)}\| \leq \frac{q}{1-q} \|\mathbf{z}^{(t)} - \mathbf{z}^{(t-1)}\| \leq \frac{q^t}{1-q} \|\mathbf{z}^{(1)} - \mathbf{z}^{(0)}\|$$

Iterating this inequality yields the stated bound on successive differences. $\square$

The parameter $q = \|\mathbf{B}\|_v$ not only guarantees convergence but also characterizes the speed of convergence. Smaller values of q imply faster stabilization of opinions, which is particularly important for large-scale networks.

The next theorem is going to be useful in showing the main result.

PROOF. Let $A \geq 0$ be the adjacency matrix and decompose it as $A = A_{\text{low}} + A_{\text{up}}$, where A_{low} and A_{up} denote the strictly lower- and upper-triangular parts, respectively. Let D be the diagonal degree matrix and define $S := I + D$, which is positive diagonal.

The iteration matrix of the FJ basic iteration is given by

$$T_{\text{FJ}} := S^{-1}A = S^{-1}(A_{\text{low}} + A_{\text{up}}),$$

while the iteration matrix corresponding to the LatestIter iteration is

$$T_{\text{LI}} := (S - A_{\text{low}})^{-1}A_{\text{up}}.$$

Since $S - A_{\text{low}}$ is lower triangular with strictly positive diagonal entries, it is invertible and $(S - A_{\text{low}})^{-1} \geq 0$. For any $x \geq 0$, define $y := T_{\text{LI}}x$. By forward substitution, y satisfies

$$y = S^{-1}(A_{\text{up}}x + A_{\text{low}}y).$$

Iterating this relation yields

$$y = \sum_{t=0}^{n-1} (S^{-1}A_{\text{low}})^t S^{-1}A_{\text{up}}x,$$

where the sum is finite because A_{low} is strictly lower triangular and $A_{\text{low}}^n = 0$.

On the other hand, for the FJ basic iteration we have

$$T_{\text{FJ}}x = S^{-1}(A_{\text{low}} + A_{\text{up}})x.$$

Since all matrices involved are nonnegative, the above expansion implies

$$T_{\text{LI}}x \leq S^{-1}(A_{\text{low}} + A_{\text{up}})x = T_{\text{FJ}}x,$$

where the inequality holds componentwise. Consequently, $0 \leq T_{\text{LI}} \leq T_{\text{FJ}}$ elementwise.

By the monotonicity of the spectral radius for nonnegative matrices, it follows that

$$\rho(T_{\text{LI}}) \leq \rho(T_{\text{FJ}}).$$

Moreover, if $A \neq 0$ and the iteration matrix T_{FJ} is irreducible (e.g., the underlying graph is connected and contains both lower- and upper-triangular couplings), then $T_{\text{LI}} \neq T_{\text{FJ}}$, and the Perron–Frobenius theorem yields the strict inequality

$$\rho(T_{\text{LI}}) < \rho(T_{\text{FJ}}).$$

Therefore, the LatestIter iteration admits a strictly smaller spectral radius than the FJ basic iteration and hence converges faster. $\square$

LEMMA 5.2 (RESIDUAL-ERROR RELATION). *Let $\mathbf{z}^{(t+1)}$ denotes the estimate updated by the latest-value iteration at epoch $t+1$, and $\mathbf{r}^{(t)} = \mathbf{s} - (\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z}^{(t)}$ denotes the residual error. Then, for all $t \geq 0$, the estimation residual between the iterate and the true solution satisfies*

$$\|\mathbf{z}^* - \mathbf{z}^{(t+1)}\|_v \leq \|(\mathbf{I} - (\mathbf{I} + \mathbf{D})^{-1}\mathbf{A})^{-1}\|_v \|\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}\|_v, \quad (8)$$

$$\begin{aligned} \mathbf{z} - \mathbf{z}^{(t)} &= (\mathbf{I} + \mathbf{L})\mathbf{r}^{(t)} \\ \mathbf{z}^{(t+1)} - \mathbf{z}^{(t)} &= (\mathbf{I} + \mathbf{D})\mathbf{r}^{(t)} \end{aligned}$$

PROOF. The latest-value update rule can be written as $\mathbf{z}^{(t+1)} = (\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}\mathbf{z}^{(t)} + (\mathbf{I} + \mathbf{D})^{-1}\mathbf{s}$. Define the successive difference $\Delta\mathbf{z}^{(t+1)} = \mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}$. Then it follows directly that $\Delta\mathbf{z}^{(t+1)} = (\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}\Delta\mathbf{z}^{(t)}$.

Since the iteration converges to the unique fixed point $\mathbf{z}^*$, we have $\mathbf{z}^* - \mathbf{z}^{(t+1)} = \sum_{k=t+1}^{\infty} \Delta\mathbf{z}^{(k+1)}$. Substituting the recursive relation of $\Delta\mathbf{z}^{(t)}$ yields

$$\begin{aligned} \mathbf{z}^* - \mathbf{z}^{(t+1)} &= \sum_{l=0}^{\infty} [(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}]^l \Delta\mathbf{z}^{(t+1)} \\ &= (\mathbf{I} - (\mathbf{I} + \mathbf{D})^{-1}\mathbf{A})^{-1} \Delta\mathbf{z}^{(t+1)}, \end{aligned} \quad (9)$$

where the inverse exists due to the convergence of the Gauss–Seidel iteration.

Taking the ℓ_1 -norm on both sides leads to

$$\|\mathbf{z}^* - \mathbf{z}^{(t+1)}\|_1 \leq \|(\mathbf{I} - (\mathbf{I} + \mathbf{D})^{-1}\mathbf{A})^{-1}\|_1 \|\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}\|_1, \quad (10)$$

which establishes the claimed relation between the true estimation error and the difference of two consecutive iterates. $\square$

Therefore, the iterative scheme yields geometrically decaying residuals, and the estimation error converges to zero. Let $q = \|\mathbf{B}\|_v$ denote a matrix norm-induced bound with $q < 1$. By Lemma,

$$\|\mathbf{z} - \mathbf{z}^{(t)}\| \leq \frac{q}{1-q} \|\mathbf{z}^{(t)} - \mathbf{z}^{(t-1)}\| \leq \frac{q^t}{1-q} \|\mathbf{z}^{(1)} - \mathbf{z}^{(0)}\|$$

Thus to achieve an absolute error $|\hat{\mathbf{z}}^{(t)} - \mathbf{z}| \leq \varepsilon \|\mathbf{z}^{(1)} - \mathbf{z}^{(0)}\|$, it suffices to take

$$T = \left\lceil \frac{\ln(\varepsilon(1-q))}{\ln(q)} \right\rceil.$$

For graphs, one can further bound γ by degree statistics: $q = \|\mathbf{B}\|_v < 1$.

Turning to efficiency and implementation locality, we summarize the per iteration cost, error decay, and neighborhood growth as follows.

THEOREM 5.3 (TIME COMPLEXITY OF LATESTITER). *To achieve an error $|\hat{\mathbf{z}}_i^{(T)} - \mathbf{z}_i| \leq \varepsilon \|\mathbf{z}\|$, the time complexity is bounded by:*

$$O\left(\min(\Theta(d_{\text{max}}^T), nT)\right),$$

$$\text{where } T = \left\lceil \frac{\log(1/\varepsilon)}{\log(1+1/d_{\text{max}})} \right\rceil.$$

PROOF. Assume $\gamma = \|(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}\|_{\infty} < 1$ and let $T = \left\lceil \frac{\log(1/\varepsilon)}{\log(1/\gamma)} \right\rceil$ be the number of iterations needed to achieve $|\hat{\mathbf{z}}_i^{(T)} - \mathbf{z}_i| \leq \varepsilon \|\mathbf{z}\|$. Then the total time to reach accuracy ε admits the following upper bounds: we can upper bound the total number of tasks in t iterations by $O(\sum_{k=0}^{t-1} |N_i(k)|) = O(\min(\Theta(d_{\text{max}}^T), nT))$. Since $\|r^{(t)}\|_{\infty}$ decays as $\|G\|_{\infty}^t$, the algorithm terminates at $\|r\|_{\infty} < \epsilon$ within at most $\ln(\epsilon)/\ln(\|(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}\|_{\infty})$ iterations. Moreover, using $\gamma \leq d_{\text{max}}(1 + d_{\text{max}})$

yields an explicit graph-parameter bound, $T \leq \left\lceil \frac{\log(1/\varepsilon)}{\log(1 + \frac{1}{d_{\text{max}}})} \right\rceil$. This completes the proof. $\square$

Algorithm 1: LatestIter for opinion estimation

Input: Graph $G = (V, E)$ with degrees (d_i) and neighbor sets (N_i) ; internal opinion $s \in \mathbb{R}^n$; maximum iteration limit $T_{\max}$; tolerance ϵ ;

Output: Estimated opinion vector z ; iteration count t

```

1  $z \leftarrow \mathbf{0} \in \mathbb{R}^n$ 
2 for  $t \leftarrow 1$  to  $T_{\max}$  do
3    $z^{\text{old}} \leftarrow z$ 
4   for  $i \leftarrow 1$  to  $n$  do
5      $h \leftarrow \mathbf{0}$ 
6     foreach  $j \in N_i$  do
7        $h \leftarrow h + \begin{cases} z_j, & j < i \\ z_j^{\text{old}}, & j > i \end{cases}$ 
8      $z_i \leftarrow (s_i + h) / (1 + d_i)$ 
9   if  $\|z - z^{\text{old}}\|_{\infty} < \epsilon$  then
10    return  $(z, t)$ 
11 return  $(z, t)$ 

```

5.3 Self-Interaction Opinion Updates

The sequential latest-opinion update captures immediate information propagation, but it implicitly assumes that agents fully replace their opinions at each update. In practice, opinion evolution is often history-dependent, where individuals adjust their current opinions rather than discard them entirely.

To model such self-interaction, we introduce a gain-controlled update that blends the previous opinion with the latest sequential estimate. Specifically, let $\tilde{z}^{(t+1)}$ denote the intermediate opinion obtained by the LatestIter scheme (defined in Section X). The final update is given by

$$z^{(t+1)} = (1 - \beta)z^{(t)} + \beta\tilde{z}^{(t+1)},$$

where $\beta \in (0, 1]$ controls the responsiveness of agents to newly propagated information.

This gain-controlled update preserves the equilibrium of the original FJ model while incorporating information from previous iterates, which enhances numerical stability and enables faster convergence in practice.

Based on this gain-controlled update rule, we propose the accelerated algorithm MEMOITER (Algorithm 2). The method introduces only a single control parameter, preserves the locality of updates, and maintains the linear structure of the original iteration. Moreover, the same gain-control principle can be naturally extended to other FJ-type iterative schemes.

Algorithm 2 is introduced as a refinement of the latest-order scheme in Algorithm 1 to improve its robustness and convergence behavior in challenging scenarios. While the latest-update iteration propagates fresh information efficiently, it may exhibit slow convergence or oscillatory behavior when the local updates are overly aggressive or when the network contains nodes with small normalization weights. To

address these limitations, MemoIter incorporates an explicit memory-based relaxation mechanism into each local update.

By interpolating between the previous iterate and the latest-order update, MemoIter retains the fast information propagation of the original scheme while significantly enhancing numerical stability and robustness.

As a result, MemoIter stabilizes the update dynamics and improves convergence reliability, especially in heterogeneous networks or near ill-conditioned regimes.

By construction, MEMOITER reduces to the LATESTITER iteration when $\beta = 1$. For undirected graphs, the associated system matrix is symmetric and positive definite, which guarantees the convergence of the scheme.

The matrix form of memory-augmented update can be written as $(\mathbf{I} + \mathbf{D})\mathbf{z}^{(k+1)} = (1 - \beta)(\mathbf{I} + \mathbf{D})\mathbf{z}^{(k)} + \beta(\mathbf{s} + \mathbf{A}_{\text{low}}\mathbf{z}^{(k+1)} + \mathbf{A}_{\text{up}}\mathbf{z}^{(k)})$. Equivalently, this can be rearranged as $\mathbf{z}^{(k+1)} = ((\mathbf{I} + \mathbf{D}) - \beta\mathbf{A}_{\text{low}})^{-1}((1 - \beta)(\mathbf{I} + \mathbf{D}) + \beta\mathbf{A}_{\text{up}})\mathbf{z}^{(k)} + \beta((\mathbf{I} + \mathbf{D}) - \beta\mathbf{A}_{\text{low}})^{-1}\mathbf{s}$, since $(\mathbf{I} + \mathbf{D}) - \beta\mathbf{A}_{\text{low}}$ is nonsingular for any admissible β .

We next analyze the convergence of the memory-augmented method and derive sufficient conditions on the parameters to ensure convergence.

LEMMA 5.4 (CONVERGENCE RANGE FOR MEMOITER). *Matrix $\mathbf{I} + \mathbf{D} - \mathbf{A}$ has nonzero diagonal entries and the memory-augmented iteration is well-defined. This method converges when $0 < \beta < 2$.*

PROOF. Suppose the memory-augmented iteration converges. Then $\rho(\mathcal{L}_{\beta}) < 1$. Let $\{\lambda_i\}_{i=1}^n$ be the eigenvalues of $\mathcal{L}_{\beta}$. By the AMGM inequality, $|\det(\mathcal{L}_{\beta})| = \prod_{i=1}^n |\lambda_i| \leq (\rho(\mathcal{L}_{\beta}))^n < 1$, hence $|\det(\mathcal{L}_{\beta})|^{1/n} < 1$.

On the other hand, using memory-augmented representation and the triangular structure,

$$\begin{aligned} \det(\mathcal{L}_{\beta}) &= \det(\mathbf{I} + \mathbf{D} - \beta\mathbf{A}_{\text{low}})^{-1} \det((1 - \beta)(\mathbf{I} + \mathbf{D}) + \beta\mathbf{A}_{\text{up}}) \\ &= \frac{1}{\prod_{i=1}^n a_{ii}} \cdot (1 - \beta)^n \prod_{i=1}^n a_{ii} = (1 - \beta)^n. \end{aligned}$$

Therefore $|(1 - \beta)^n|^{1/n} = |1 - \beta| < 1$, which is equivalent to $0 < \beta < 2$.

For the converse, when $\mathbf{I} + \mathbf{D} - \mathbf{A}$ is strictly diagonally dominant by rows or SPD, classical theory of stationary iterations implies that the iteration matrix has spectral radius strictly smaller than one. The iteration matrix can be written as $\mathcal{L}_{\beta} = (\mathbf{I} - \beta(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}_{\text{low}})^{-1}((1 - \beta)(\mathbf{I} + \mathbf{D}) + \beta(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}_{\text{up}})$, and for $0 < \beta < 2$ it is a convergent regular splitting of $\mathbf{I} - \beta(\mathbf{I} + \mathbf{D})^{-1}\mathbf{A}$. Under strict diagonal dominance or SPD, this yields $\rho(\mathcal{L}_{\beta}) < 1$. Hence the memory-augmented iteration converges. $\square$

When $\beta \in (0, 1)$, the update corresponds to a conservative correction, where opinions are adjusted cautiously toward the latest estimate. The case $\beta = 1$ recovers the sequential latest-opinion updates, while $\beta \in (1, 2)$ induces an over-corrective behavior that can further accelerate convergence by intentionally amplifying the correction step. Accordingly, the range $\beta \in (0, 2)$ serves as a sufficient condition

that ensures convergence while maintaining a consistent interpretation of opinion adjustment.

referred to as the *adjustment gain*, regulates the magnitude of correction applied toward the latest locally computed opinion estimate. views it as a control parameter governing the responsiveness of opinion updates. The final opinion update is then obtained by applying a gain-controlled correction to the current opinion:

$$\mathbf{z}^{t+1} = (1 - \beta)\mathbf{z}^{(t)} + \beta\tilde{\mathbf{z}}^{(t+1)} = \mathbf{z}^{(t)} + \beta(\tilde{\mathbf{z}}^{(t+1)} - \mathbf{z}^{(t)}),$$

This enables a larger amount of residual mass to be transferred in each update, thereby accelerating convergence while maintaining locality.

Moreover, we derive the optimal choice of β that minimizes the convergence rate of the memory-augmented algorithm.

LEMMA 5.5 (OPTIMAL CONVERGENT PARAMETER). *Matrix $\mathbf{I} + \mathbf{D} - \mathbf{A}$ is SPD and let $B_0 = (\mathbf{I} + \mathbf{D})^{-1}(\mathbf{A}_{low} + \mathbf{A}_{up})$ be the iteration matrix with spectral radius $\rho(B_0) = \rho \in (0, 1)$. Then the spectral radius of the iteration matrix satisfies*

$$\rho(\mathcal{L}_\beta) = \left| \frac{1 - \sqrt{1 - \rho^2}}{1 + \sqrt{1 - \rho^2}} \right| \quad \text{when} \quad \beta = \beta^* \triangleq \frac{2}{1 + \sqrt{1 - \rho^2}}.$$

Moreover, $\beta^* \in (1, 2)$ and minimizes $\rho(\mathcal{L}_\beta)$ over $\beta \in (0, 2)$.

PROOF. For iterative matrix, B_0 is diagonalizable with real spectrum in $(-1, 1)$, and the memory-augmented and latest-value iterations are related through Young's theory of semi-iterative methods. In particular, there exists a one-to-one correspondence between each eigenvalue $\mu \in (-1, 1)$ of B_0 and an eigenvalue $\lambda(\omega, \mu)$ of $\mathcal{L}_\omega$ given by the quadratic relation $\lambda^2 - (1 - \omega)\lambda - \omega^2 \frac{\mu^2}{4} = 0$. Solving (5.3) yields $\lambda_{\pm}(\omega, \mu) = \frac{1}{2}(1 - \omega \pm \sqrt{(1 - \omega)^2 + \omega^2 \mu^2})$. Hence the spectral radius of $\mathcal{L}_\omega$ is $\rho(\mathcal{L}_\omega) = \max_{\mu \in [-\rho, \rho]} \max\{|\lambda_+(\omega, \mu)|, |\lambda_-(\omega, \mu)|\}$. For fixed $\omega \in (0, 2)$, the right-hand side is maximized at the extreme values $\mu = \pm\rho$. Therefore the best ω solves

$$\min_{\omega \in (0, 2)} \phi_\rho(\omega) \triangleq \frac{1}{2} \left| 1 - \omega + \sqrt{(1 - \omega)^2 + \omega^2 \rho^2} \right|.$$

Elementary calculus (setting $\frac{d}{d\omega}\phi_\rho(\omega) = 0$ for $\omega \in (0, 2)$) or the standard Chebyshev argument gives the minimizer $\omega^* = \frac{2}{1 + \sqrt{1 - \rho^2}}$, for which the two roots have equal magnitude, and the resulting minimal spectral radius is $\rho(\mathcal{L}_\omega)|_{\omega=\omega_{opt}} = \frac{1 - \sqrt{1 - \rho^2}}{1 + \sqrt{1 - \rho^2}} \in (0, 1)$. Since $\rho \in (0, 1)$, we have $\sqrt{1 - \rho^2} \in (0, 1)$ and thus $\omega_{opt} \in (1, 2)$. This completes the proof. $\square$

6 Opinion Iteration with Historical Memory

In the previous section, we developed accelerated opinion update schemes by refining the iterative perspective of the classical Friedkin–Johnsen (FJ) model. In particular, the *LatestIter* framework exploits the most recently available neighbor information, while the gain-controlled variant further

Algorithm 2: GainIter (Gain-controlled latest-order opinion iteration)

Input: Graph $G = (V, E)$ with degrees (d_i) and neighbor sets (N_i) ; internal opinion $s \in \mathbb{R}^n$; relaxation parameter $\beta > 0$; maximum iteration limit T_{max} ; tolerance ϵ

Output: Estimated opinion vector $\mathbf{z}$; iteration count $\mathbf{t}$

```

1  $\mathbf{z} \leftarrow \mathbf{0} \in \mathbb{R}^n$ ;  $\mathbf{z}^{\text{old}} \leftarrow \mathbf{z}$ ;  $\alpha_i \leftarrow 1/(1 + d_i)$  for  $i = 1, \dots, n$ 
2 for  $\mathbf{t} \leftarrow 1$  to  $T_{max}$  do
3    $\mathbf{z}^{\text{old}} \leftarrow \mathbf{z}$ 
4   for  $i \leftarrow 1$  to  $n$  do
5      $\mathbf{h} \leftarrow \mathbf{0}$ 
6     foreach  $j \in N_i$  do
7        $\mathbf{h} \leftarrow \mathbf{h} + a_{ij} \cdot (\text{Latest}(j))$ 
7       // Latest( $j$ ):  $z_j$  if node  $j$  has been
7       updated, otherwise  $z_j^{\text{old}}$ .
8      $y_i \leftarrow \alpha_i (s_i + \mathbf{h})$ 
9      $z_i \leftarrow z_i + \beta (y_i - z_i)$ 
10  if  $\|\mathbf{z} - \mathbf{z}^{\text{old}}\|_\infty \leq \epsilon$  then
11    return  $(\mathbf{z}, \mathbf{t})$ 
12 return  $(\mathbf{z}, \mathbf{t})$ 
```

modulates the relative impact of newly propagated opinions and agents' current states. These designs are motivated by practical considerations in real-world opinion diffusion and remain within the standard first-order dynamical setting, where opinion evolution depends solely on the current configuration.

Beyond the iterative interpretation, the FJ model admits an alternative and equally important explanation from a game-theoretic perspective. Specifically, it can be viewed as a quadratic utility game in which each agent balances social conformity against attachment to its intrinsic opinion, and the resulting equilibrium corresponds to a Nash equilibrium that minimizes a global social cost. This viewpoint highlights the FJ dynamics as an implicit equilibrium-seeking process driven by rational, yet bounded, decision making.

However, existing studies predominantly focus on a Markovian opinion updating mechanism, where the state transition depends only on the current opinion profile. Such memoryless dynamics neglect the influence of historical opinions, despite the fact that, in realistic social interactions, individuals often revise their beliefs by accounting for recent experience and accumulated past stances. From a game-theoretic standpoint, this limitation excludes an important class of path-dependent behaviors that naturally arise in repeated or evolving games, especially under noisy environments or gradual belief stabilization.

Motivated by this observation, we extend the FJ opinion dynamics by incorporating a finite memory into the update process. Rather than relying exclusively on the current opinion state, the proposed model allows agents to leverage their

immediate opinion history, leading to a second-order opinion iteration. This modification preserves the linear structure of the original FJ model, while enriching its temporal dynamics and enabling a more faithful representation of history-dependent strategic adjustment. As will be shown, the resulting scheme admits a compact state-space formulation, which facilitates convergence analysis and spectral characterization.

While this interpretation justifies the convergence of standard FJ dynamics, it also reveals an inherent limitation: the induced best-response process is memoryless and depends only on the current opinion profile. In contrast, many repeated and evolving games exhibit path-dependent behavior, where agents incorporate past actions to stabilize or accelerate convergence. This observation motivates the incorporation of historical opinions into the FJ dynamics, leading naturally to a second-order, memory-augmented opinion evolution scheme.

Recall that the classical FJ iteration updates the opinion vector according to $\mathbf{z}^{(t+1)} = (\mathbf{I} + \mathbf{D})^{-1} \mathbf{A} \mathbf{z}^{(t)} + (\mathbf{I} + \mathbf{D})^{-1} \mathbf{s}$, where $\mathbf{z}^{(t)} = [z_1^{(t)}, \dots, z_n^{(t)}]^\top$ denotes the opinion profile at iteration t .

In the memory-augmented setting, given the current and previous opinion states $\mathbf{z}^{(t)}$ and $\mathbf{z}^{(t-1)}$, we first compute the intermediate updates produced by the FJ operator, $\mathbf{x}^{(t)} = (\mathbf{I} + \mathbf{D})^{-1} \mathbf{A} \mathbf{z}^{(t)} + (\mathbf{I} + \mathbf{D})^{-1} \mathbf{s}$, $\mathbf{x}^{(t-1)} = (\mathbf{I} + \mathbf{D})^{-1} \mathbf{A} \mathbf{z}^{(t-1)} + (\mathbf{I} + \mathbf{D})^{-1} \mathbf{s}$. The next opinion state is then obtained by forming a convex combination of these two intermediate states,

$$\mathbf{z}^{(t+1)} = \eta \mathbf{x}^{(t)} + (1 - \eta) \mathbf{x}^{(t-1)},$$

where $\eta \in (0, 1)$ controls the relative influence of the most recent and historical information. The initial condition is set as $\mathbf{z}^{(0)} = \mathbf{z}^{(-1)}$.

From an implementation perspective, this scheme requires each agent to retain only the locally aggregated opinion from the previous iteration, rather than the entire opinion trajectory. As a result, the memory overhead remains minimal while enabling richer temporal dynamics. When $\eta = 1$, the iteration reduces to the standard FJ model.

To facilitate analysis, the memory-augmented system can be equivalently expressed in a first-order state-space form. Define

$$\mathbf{u}^{(t)} = \mathbf{z}^{(t-1)}, \quad \mathbf{y}^{(t)} = \begin{bmatrix} \mathbf{z}^{(t)} \\ \mathbf{u}^{(t)} \end{bmatrix}.$$

Then the evolution of $\mathbf{y}^{(t)}$ is governed by

$$\mathbf{y}^{(t+1)} = \mathbf{G} \mathbf{y}^{(t)} + \mathbf{c},$$

where

$$\mathbf{G} = \begin{bmatrix} (\mathbf{I} + \mathbf{D})^{-1} \mathbf{A} & (1 - \eta)(\mathbf{I} + \mathbf{D})^{-1} \mathbf{A} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} (\mathbf{I} + \mathbf{D})^{-1} \mathbf{s} \\ \mathbf{0} \end{bmatrix}.$$

The convergence of the proposed second-order iteration is therefore characterized by the spectral radius $\rho(\mathbf{G})$, which determines the worst-case convergence rate and forms the basis for the subsequent theoretical analysis.

THEOREM 6.1. *Let $\mathbf{A}$ be an averaging matrix with a real spectrum. Suppose that $x_i(1) = y_i$, $i \in \{1, 2, \dots, n\}$, and*

$\sum_{i=1}^n x_i(0) = \sum_{i=1}^n y_i$. Then the opinion vector $\mathbf{z}(t)$ of the second-order iteration converges, and faster than the original FJ iteration if $0 < \gamma < \sigma_2^2$.

PROOF OF THEOREM 1 (ITEM 4). Let $\mathbf{A}$ be an averaging matrix with real spectrum, and consider the augmented iteration matrix

$$\mathbf{B} = \begin{bmatrix} (g+1)\mathbf{A} & -g\mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}, \quad -1 < g < 1.$$

Denote by $\rho_2(\mathbf{B})$ the convergence factor of the augmented system (4), and by $\sigma_2 \in [0, 1]$ the second largest singular value of $\mathbf{A}$, which is also the convergence factor of the original system (2). Item 4 asserts that the augmented system converges strictly faster than the original one whenever

$$0 < g < \sigma_2^2, \quad \text{i.e.,} \quad \rho_2(\mathbf{B}) < \sigma_2.$$

We first consider the case $-1 < g \leq g^*$. By Lemma 4, the convergence factor of the augmented system is given by

$$\rho_2(\mathbf{B}) = \frac{1}{2} \left(\sigma_2(g+1) + \sqrt{\sigma_2^2(g+1)^2 - 4g} \right),$$

and in particular $\rho_2(\mathbf{B}) = \sigma_2$ when $g = 0$. Moreover, Proposition 3 shows that $\rho_2(\mathbf{B})$ is strictly decreasing on the interval $(-1, g^*]$. Consequently, for any

$$0 < g \leq \min\{g^*, \sigma_2^2\},$$

we have

$$\rho_2(\mathbf{B}) < \rho_2(\mathbf{B})|_{g=0} = \sigma_2.$$

Next, consider the case $g^* < g < 1$. By Lemma 5, the convergence factor simplifies to

$$\rho_2(\mathbf{B}) = \sqrt{g}.$$

Therefore, for any g satisfying

$$g^* < g < \sigma_2^2,$$

it follows immediately that

$$\rho_2(\mathbf{B}) = \sqrt{g} < \sigma_2.$$

Combining the two cases above, we conclude that for all

$$0 < g < \sigma_2^2,$$

the inequality

$$\rho_2(\mathbf{B}) < \sigma_2$$

holds. Hence, the augmented system (4) converges strictly faster than the original system (2), which completes the proof of Item 4. $\square$

PROPOSITION 6.2. *Suppose that $\mathbf{A}$ is an matrix with a real spectrum. Let*

$$\mathbf{B} = \begin{bmatrix} (\gamma+1)\mathbf{A} & -\gamma\mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}$$

where $-1 < \gamma < 1$. Then the minimum of $\rho_2(\mathbf{B})$ is unique. The iteration converge fastest when

$$\gamma = \frac{1 - \sqrt{1 - \sigma_2^2}}{1 + \sqrt{1 - \sigma_2^2}}. \quad (11)$$

PROOF OF PROPOSITION 3. From Lemmas 4 and 5, it is clear that $\rho_2(B)$ is a continuous function of γ on the interval $(-1, 1)$. When $-1 < \gamma \leq \gamma^*$, by Lemma 4 and the fact that $\partial \rho_2 / \partial \gamma < 0$, $\rho_2(B)$ is a decreasing function of γ . When $\gamma^* < \gamma < 1$, $\rho_2(B)$ is an increasing function of γ by Lemma 5. Since $\rho_2(B)$ is continuous at γ^* , γ^* is the unique point in the interval $(-1, 1)$ which minimizes $\rho_2(B)$. Then the optimal value of γ at this minimum is

$$\gamma^* = \frac{1 - \sqrt{1 - \sigma_2^2}}{1 + \sqrt{1 - \sigma_2^2}},$$

and the minimum of $\rho_2(B)$ is

$$\rho_2^* = \frac{\sigma_2}{1 + \sqrt{1 - \sigma_2^2}}. \quad (12)$$

□

Algorithm 3: SECONDORDERITER($G, \gamma, \epsilon, T_{max}$)

Input : A graph $G = (V, E)$ with adjacency matrix A , degree vector d , internal opinions s ; acceleration parameter $\gamma \in \mathbb{R}$; tolerance ϵ and maximum iterations T_{max} .

Output: Estimated opinion vector z .

1 **Precompute:**

2 $\hat{D}^{-1} \leftarrow \text{diag}\left(\frac{1}{1 + d_i}\right)_{i \in V}$; $P \leftarrow \hat{D}^{-1}A$; $b \leftarrow \hat{D}^{-1}s$

3 **Initialize:**

4 $z^{(0)} \leftarrow s$; $k \leftarrow 0$; $r \leftarrow +\infty$

5 **while** $k < T_{max}$ **and** $r > \epsilon$ **do**

6 $u \leftarrow P z^{(k)}$ // u is a work vector

7 $z^{(k+1)} \leftarrow \gamma u + (1 - \gamma) z^{(k-1)} + b$

8 $r \leftarrow \|z^{(k+1)} - z^{(k)}\|_2$

9 **if** $r \leq \epsilon$ **then**

10 $\quad \text{return } z^{(k+1)}$

11 $z^{(k-1)} \leftarrow z^{(k)}$; $z^{(k)} \leftarrow z^{(k+1)}$

12 $k \leftarrow k + 1$

13 **return** $z^{(k)}$

7 Experiments

In this section, we assess the efficiency and accuracy of our estimation algorithm LATESTITER and MEMOITER. To this end, we implement this algorithm on various real networks and compare the running time and accuracy of our algorithm with those corresponding to the previous algorithm, called LOCALITER. The estimation of relevant quantities perform product of related matrices, and calculate corresponding ℓ_2 norms.

Environment. Our code for the estimation algorithm LATESTITER and MEMOITER were written in *Julia v1.5.1*. The

Julia language implementation for which is open and accessible on the website¹.

Datasets and Equipment All the real-world networks we consider are publicly available in the Koblenz Network Collection and Network Repository. Our experiments are conducted on a diverse range of both undirected and directed networks, including social networks. For these datasets, the number of nodes ranges from about 16 thousand to 33 million, and the number of directed edges ranges from about 25 thousand to 301 million. The details of these datasets are presented in Table 1. All experiments are conducted using the Julia programming language. We conduct all experiments in a computational environment featuring a 4.2 GHz Intel i7-7700 CPU with 64GB of primary memory.

Algorithm For evaluating the performance of our algorithms estimating the opinion vector of the FJ model, we compare our three proposed algorithms LATESTITER, MEMOITER, and SECONDITER with two state-of-the-art algorithms, namely SOLVER and BLI SOR. While one of the two state-of-the-art algorithms, SOLVER, is confined to undirected graphs due to the limitations of the Laplacian solver. Our proposed algorithm are applicable to both undirected and directed graphs.

Internal opinion distributions. In our experiments, the internal opinions are generated according to three different distributions: uniform distribution, exponential distribution, and power-law distribution. For the uniform distribution, the opinion s_i of node i is distributed uniformly in the range of $[0, 1]$. For the exponential distribution, we choose the probability density $e^{x_{\min}} e^{-x}$ to generate n' positive numbers x with minimum value $x_{\min} > 0$. Then, dividing each x by the maximum observed value, we normalize these n' numbers to the range $[0, 1]$ as the internal opinions of nodes. Note that there is always a node with internal opinion 1 due to the normalization operation. Similarly, for the power-law distribution, we use the probability density $(\alpha - 1)x_{\min}^{\alpha-1} x^{-\alpha}$ with $\alpha = 2.5$ to generate n' positive numbers, and normalize them to interval $[0, 1]$ as the internal opinions.

7.1 Experiments on Undirected graph

Efficiency. Table 3 reports the running time of six methods, namely SOLVER, LI, LISOR, LATESTITER, MEMOITER, and SECONDITER, on several real-world networks of increasing scale. We observe that SOLVER, which computes the exact solution by directly solving the linear system, is consistently the most time-consuming method and becomes increasingly expensive as the network size grows. In contrast, all iterative methods exhibit significantly better scalability.

Among the iterative solvers, LI and LISOR show stable and predictable performance across all datasets, with LISOR being slightly faster than LI in most cases. The Gauss-Seidel based method LATESTITER and its over-relaxed variant MEMOITER further reduce the running time on medium-scale networks such as Enron and DBLP, while maintaining low iteration counts. The second-order accelerated method

¹URL omitted here.

Table 1: Statistics of the evaluated datasets.

Dataset	n	m	d_{max}
BlogCatalog	2,426	16,630	273
DBLP	317,080	1,049,866	343
YouTubeSnap	1,134,890	2,987,624	28,754
Flixster	2,523,386	7,918,801	1,474
out.youtube-u-growth	3,072,441	117,185,083	33,313

SECONDITER is the fastest on several large graphs, including YouTubeSnap and out.youtube, where it achieves the shortest running time among all tested methods.

Overall, Table 3 demonstrates that the proposed iterative methods provide substantial efficiency gains over the exact solver, especially on large-scale graphs with millions of nodes, where SOLVER becomes prohibitively slow, while the iterative methods remain practical.

Accuracy. Besides efficiency, we also evaluate the accuracy of the iterative methods by comparing their outputs with the exact solution produced by SOLVER. For each method and each dataset, we report the absolute error on a representative entry of the solution vector, as shown in the right part of Table 3.

The results indicate that LI and LISOR consistently achieve errors on the order of 10^{-3} or below across all datasets, demonstrating reliable approximation quality. LATESTITER and MEMOITER generally attain even smaller errors, often in the range of 10^{-5} to 10^{-4} , especially on moderately sized networks such as Enron and DBLP. Although SECONDITER occasionally exhibits slightly larger errors on some large graphs (e.g., DBLP and out.youtube), its error remains within acceptable bounds for practical applications.

Overall, the empirical results confirm that all iterative methods achieve high accuracy in practice, with errors that are small

7.2 Experiments on directed graph

8 Conclusion

In this paper, we addressed the problem of computing single-node equilibrium opinions in the Friedkin-Johnsen model. We introduced the absorbing random-walk interpretation and developed local algorithms that avoid full-matrix computation. Our first contribution is a residual-based framework with synchronous and asynchronous iterations, together with a residue invariant that precisely characterizes the error and enables practical stopping rules and convergence bounds.

To further enhance scalability, we proposed a push-sampling algorithm (AIMS) that couples asynchronous local pushes with importance sampling over the residual distribution. The method preserves the invariant, uses random walks only to estimate the unprocessed mass, and yields an unbiased estimator with concentration guarantees. The τ -controlled sampling budget provides a clear trade-off between push work

Algorithm 4: Fixed-point iteration for $z = (I + D)^{-1}(s + Az)$

Input: Sparse $A \in \mathbb{R}^{n \times n}$, $s \in \mathbb{R}^n$, optional z_0 , tolerance $\varepsilon > 0$, maxiter, relaxation $\lambda \in (0, 2]$

Output: z , iteration count k , convergence flag

```

1  denomi ← 1 + ∑j Aij for  $i = 1, \dots, n$ 
   //  $I + D = \text{diag}(\text{denom})$ 
2   $z \leftarrow s$ ,
3  for  $k \leftarrow 1$  to maxiter do
4       $a \leftarrow Az$  // sparse matvec
5       $z_i^{\text{new}} \leftarrow (s_i + a_i) / \text{denom}_i$  for  $i = 1, \dots, n$ 
6      if  $\lambda \neq 1$  then
7           $z^{\text{new}} \leftarrow (1 - \lambda)z + \lambda z^{\text{new}}$  // relaxation
8      if  $\|z^{\text{new}} - z\|_{\infty} < \varepsilon$  then
9          return ( $z^{\text{new}}$ ,  $k$ , true)
10      $z \leftarrow z^{\text{new}}$ 
11 return ( $z$ , maxiter, false)

```

and variance, making the approach effective on both directed and undirected graphs.

Extensive experiments on real networks with up to tens of millions of nodes demonstrate that our methods are accurate and efficient, outperforming classical solvers and direct Monte Carlo baselines, and extending naturally to average-opinion estimation.

Currently, our framework does not adaptively schedule the mix of push and sampling across nodes. In future work, we plan to develop variance-aware scheduling, extend to time-varying and weighted networks, and integrate our methods with preconditioned linear solvers to support batch queries.

A Appendix: Pseudocode

In Appendix A, we present eliminated pseudocode in the main text.

A.1 Pseudocode of WeightedForest

B Appendix: Proof

In Appendix B, we provide the proofs of the theorems and lemmas in the main text.

B.1 Useful Tools

B.1.1 (matrix convergence).

LEMMA B.1. Let $B = (b_{ij})_{n \times n}$. Then $B^k \rightarrow 0$ as $k \rightarrow \infty$ if and only if $\rho(B) < 1$.

B.2 Proof of Equivalence with Jacobi

To show that FJ Iteration is a variant of the fixed-point iteration, recall that for solving the linear system $M\mathbf{x} = \mathbf{b}$, the Jacobi method updates the solution vector $\mathbf{x}$. Specifically,

Table 2: Datasets and comparison of running time (seconds) and error between RWB and Asy.

Dataset	Time (s)						Error					
	RWB			Asy			RWB			Asy		
	normal	Exp	Power-law	Uniform	Exp	poisson	Uniform	Exp	poisson	Uniform	Exp	poisson
BlogCatalog	3.13	2.63	3.10	0.80	0.73	0.86	3.16e-4	1.5e-4	1.73e-4	4.99e-7	4.98e-7	4.95e-7
DBLP	5.51	5.19	4.90	1.31	1.13	1.85	4.42e-5	9.13e-5	3.45e-4	5.16e-7	5.18e-7	5.12e-7
YouTubeSnap	8.11	7.63	8.10	2.81	2.50	1.35	3.77e-4	2.79e-4	2.18e-4	5.96e-7	5.86e-7	5.93e-7
Flixster	13.67	13.19	13.3	11.86	9.69	12.84	1.51e-5	8.81e-7	4.06e-4	6.10e-7	6.10e-7	6.11e-7
out.youtube	19.37	19.62	18.7	19.65	15.15	21.46	2.23e-4	2.55e-5	4.21e-4	4.92e-7	5.05e-7	4.96e-7

Table 3: Running time (seconds) and error of different iterative methods on real-world networks.

Dataset	RUNNING TIME (s)						ERROR				
	Solver	LI	LISOR	LatestIter	MemoIter	SecondIter	LI	LISOR	LatestIter	MemoIter	SecondIter
BlogCatalog	5.08	0.54	0.20	0.08	0.02	0.47	1.29×10^{-3}	5.62×10^{-4}	3.44×10^{-2}	7.09×10^{-2}	1.38×10^{-3}
Enron	0.37	0.02	0.01	0.03	0.01	0.01	3.13×10^{-1}	3.12×10^{-1}	5.11×10^{-3}	1.78×10^{-3}	8.74×10^{-2}
DBLP	2.37	0.66	0.16	0.39	0.09	0.16	1.28×10^{-3}	4.79×10^{-4}	3.19×10^{-5}	8.76×10^{-5}	2.24×10^{-2}
YouTubeSnap	7.19	2.23	0.56	0.65	0.35	0.21	1.30×10^{-3}	2.74×10^{-4}	1.78×10^{-3}	8.43×10^{-4}	1.44×10^{-2}
Flixster	17.41	7.64	1.88	1.72	0.94	3.19	1.35×10^{-3}	2.97×10^{-4}	3.08×10^{-3}	3.35×10^{-4}	3.89×10^{-3}
out.youtube	26.64	12.02	2.33	3.75	1.39	0.84	1.09×10^{-3}	5.35×10^{-5}	3.46×10^{-4}	5.13×10^{-4}	8.37×10^{-3}

the update for coordinate u at iteration t is

$$x_u^{t+1} = \frac{1}{m_{uu}} \left(b_u - \sum_{j \neq i} m_{uj} x_j^t \right), \quad (13)$$

where t indexes the current iteration. Here, x_j^t denotes entries updated in the t -th sweep. In the classical Jacobi method we simply update all coordinates during iteration t .

The iteration is well suited for linear systems where $\mathbf{M}$ is strictly diagonally dominant. In particular, for a simple graph, the matrix $\mathbf{I} + \mathbf{D} - \mathbf{A}$ is strictly diagonally dominant, and thus its fixed-point iteration naturally fits into the Jacobi framework.

The theorem below establishes the precise equivalence between this variant of the fixed-point iteration and the FJ Iteration.

LEMMA B.2 (FIXED-POINT FORM OF THE FJ ITERATION). *The FJ iteration in (1) is a fixed-point iteration of the affine mapping*

$$T(z) := (\mathbf{I} + \mathbf{D})^{-1}(\mathbf{s} + \mathbf{A}z),$$

whose fixed point is the unique solution of the linear system

$$(\mathbf{I} + \mathbf{D} - \mathbf{A})z = \mathbf{s}.$$

PROPOSITION B.3 (EQUIVALENCE WITH JACOBI COORDINATE UPDATES). *Under a sequential coordinate sweep, each coordinate update of the FJ iteration coincides exactly with a Jacobi update defined in 13 for the linear system*

$$\mathbf{M}z = \mathbf{b}, \quad \mathbf{M} = \mathbf{I} + \mathbf{D} - \mathbf{A}, \quad \mathbf{b} = \mathbf{s}.$$

PROOF. Denote the approximate solution vector $\mathbf{z}_i^{(t)}$ defined by $\mathbf{z}_i^{(t)} = (z_1^{(t)}, z_2^{(t)}, \dots, z_{i-1}^{(t)}, z_i^{(t-1)}, \dots, z_n^{(t-1)})$. Set the

residual

$$\mathbf{r}^{(t)} := \mathbf{b} - \mathbf{M}\mathbf{z}^{(t)} = \mathbf{s} - (\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z}^{(t)}.$$

Using the FJ update we obtain the linear invariant

$$\begin{aligned} \mathbf{z}^{(t+1)} - \mathbf{z}^{(t)} &= (\mathbf{I} + \mathbf{D})^{-1}(\mathbf{s} + \mathbf{A}\mathbf{z}^{(t)}) - \mathbf{z}^{(t)} \\ &= (\mathbf{I} + \mathbf{D})^{-1}[\mathbf{s} - (\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z}^{(t)}] \\ &= (\mathbf{I} + \mathbf{D})^{-1}\mathbf{r}^{(t)}. \end{aligned}$$

Equivalently, $\mathbf{r}^{(t)} = (\mathbf{I} + \mathbf{D})(\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)})$.

Let u be the coordinate being updated at time t in a G-S sweep. For the system $\mathbf{M}z = \mathbf{b}$ the G-S coordinate update reads

$$z_u = \frac{1}{M_{uu}}(b_u - \sum_{j \neq u} M_{uj} z_j^{(t)}),$$

subtract $\mathbf{z}_i^{(t)}$ from both sides,

$$z_u^{(t+1)} = z_u^{(t)} + \frac{(\mathbf{b} - \mathbf{M}\mathbf{z}^{(t)})_u}{M_{uu}} = z_u^{(t)} + \frac{r_u^{(t)}}{M_{uu}}.$$

Because $\mathbf{M} = (\mathbf{I} + \mathbf{D} - \mathbf{A})$, its diagonal is

$$M_{uu} = (\mathbf{I} + \mathbf{D} - \mathbf{A})_{uu} = 1 + d_u.$$

Hence

$$z_u^{(t+1)} = z_u^{(t)} + \frac{r_u^{(t)}}{1 + d_u},$$

which is equivalent to the above invariants by expanding the FJ update for coordinate u :

$$z_u^{(t+1)} = z_u^{(t)} + ((\mathbf{I} + \mathbf{D})^{-1}\mathbf{r}^{(t)})_u = z_u^{(t)} + \frac{r_u^{(t)}}{1 + d_u},$$

It matches exactly the Gauss-Seidel coordinate update. Since this holds for any updated coordinate u , each coordinate update of the FJ iteration is equivalent to a single Gauss-Seidel update for the linear system $\mathbf{M}z = \mathbf{b}$. $\square$

FJ Iteration is actually a variant of the fixed-point iteration,

LEMMA B.4 (FIXED-POINT FORM OF THE FJ ITERATION). *The FJ iteration in (1) is a fixed-point iteration of the affine mapping*

$$T(z) := (\mathbf{I} + \mathbf{D})^{-1}(\mathbf{s} + \mathbf{A}z),$$

whose fixed point is the unique solution of the linear system

$$(\mathbf{I} + \mathbf{D} - \mathbf{A})z = \mathbf{s}.$$

LEMMA B.5. *Throughout the iterations of the Gauss–Seidel–equivalent scheme, the following invariant relation is preserved:*

$$\mathbf{z}^* - \mathbf{z}^{(t)} = (\mathbf{I} + \mathbf{L})^{-1} \mathbf{r}^{(t)}, \quad (14)$$

where $\mathbf{r}^{(t)} = \mathbf{s} - (\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z}^{(t)}$ denotes the error vector measuring the deviation of the t -th iterate from the true solution $\mathbf{z}^*$.

PROOF. By definition, the exact solution satisfies $\mathbf{z}^* = (\mathbf{I} + \mathbf{D} - \mathbf{A})^{-1} \mathbf{s}$. Subtracting $\mathbf{z}^{(t)}$ from both sides yields

$$\begin{aligned} \mathbf{z}^* - \mathbf{z}^{(t)} &= (\mathbf{I} + \mathbf{D} - \mathbf{A})^{-1} \mathbf{s} - \mathbf{z}^{(t)} \\ &= (\mathbf{I} + \mathbf{D} - \mathbf{A})^{-1} (\mathbf{s} - (\mathbf{I} + \mathbf{D} - \mathbf{A})\mathbf{z}^{(t)}) \\ &= (\mathbf{I} + \mathbf{D} - \mathbf{A})^{-1} \mathbf{r}^{(t)}. \end{aligned} \quad (15)$$

Since $\mathbf{L} = \mathbf{D} - \mathbf{A}$, the claimed invariant relation follows. Therefore, throughout the Gauss–Seidel–equivalent iterations, the error between the current iterate and the true solution is completely characterized by the residual vector $\mathbf{r}^{(t)}$, and the stated invariant is preserved. $\square$

Based on the invariant established in Lemma 1, we can further derive an absolute error bound for the estimator upon termination of the iteration.

THEOREM B.6 (ERROR BOUND). *For any accuracy parameter $\epsilon > 0$, the estimator $\mathbf{z}^{(t)}$ returned by the algorithm satisfies*

$$\|\mathbf{z}^* - \mathbf{z}^{(t)}\|_2 \leq \epsilon. \quad (16)$$

PROOF. From Lemma 1, we have $\mathbf{z}^* - \mathbf{z}^{(t)} = (\mathbf{I} + \mathbf{L})^{-1} \mathbf{r}^{(t)}$. Taking the ℓ_2 -norm on both sides gives

$$\begin{aligned} \|\mathbf{z}^* - \mathbf{z}^{(t)}\|_2 &= \|(\mathbf{I} + \mathbf{L})^{-1} \mathbf{r}^{(t)}\|_2 \\ &\leq \|(\mathbf{I} + \mathbf{L})^{-1}\|_2 \|\mathbf{r}^{(t)}\|_2. \end{aligned} \quad (17)$$

By the termination criterion of the algorithm, the residual norm satisfies $\|(\mathbf{I} + \mathbf{L})^{-1}\|_2 \|\mathbf{r}^{(t)}\|_2 \leq \epsilon$, which completes the proof. $\square$

Nesterov’s Accelerated Gradient (NAG) Method. Given a strongly convex function f , NAG method, at t -th iteration, updates two vectors $\mathbf{y}^t$ and $\mathbf{z}^t$. It updates $\mathbf{y}^t$ with the help of $\mathbf{z}^t$ (with $\mathbf{z}^0 = \mathbf{y}^0$) as the following:

$$\mathbf{z}^t = \mathbf{y}^{t-1} - \eta_t \nabla f(\mathbf{y}^{t-1}), \quad \mathbf{y}^t = \mathbf{z}^t + \beta_t (\mathbf{z}^t - \mathbf{z}^{t-1}).$$

If we set $\eta_t = 1/(2 - \alpha)$ and $\beta_t = (1 - \kappa)/(1 + \kappa)$ with the inverse square root of condition number

$$\kappa = \sqrt{(2 - \alpha)/\alpha},$$

then we can write the iteration in one line as the following iteration procedure:

$$\text{NAG: } \mathbf{y}^{t+1} = 2\left[\eta(\mathbf{I} + \tilde{P})\mathbf{y}^t\right] - (1 - \kappa)\left[\eta(\mathbf{I} + \tilde{P})\mathbf{y}^{t-1}\right] + \kappa^2 \tilde{\mathbf{s}}. \quad (18)$$

where $\eta = (1 - \alpha)/((2 - \alpha)(1 + \kappa))$ and $\mathbf{y}_0 = 0$, $\mathbf{y}_1 = \alpha D^{-1/2} \mathbf{e}_s$. Notice that $(\mathbf{I} + \tilde{P})\mathbf{y}^t$ can be used for the next iteration; hence the per-iteration operations are at most m . Then we have the following convergence rate for ℓ_1 estimation error of $\mathbf{x}^*$.

THEOREM B.7. *Let $\mathbf{y}^{t+1}$ be the estimated vector returned by NAG method using iteration (18) (with $\mathbf{z}^0 = \mathbf{y}^0 = 0$) and let $\mathbf{x}^t = D^{1/2} \mathbf{y}^t$, then the estimation error of $\mathbf{x}^*$ is upper bounded by*

$$\|\mathbf{x}^t - \mathbf{x}^*\|_1 \leq d_{\max} \sqrt{\frac{2n}{\alpha}} \exp\left(-\frac{t-1}{2\sqrt{(2-\alpha)/\alpha}}\right). \quad (19)$$

And the total number of operations required for ϵ -precision of per-entry error $\|\mathbf{x}^t - \mathbf{x}^*\|_\infty \leq \epsilon$ is

$$R_T = O\left(\frac{m}{\sqrt{\alpha}} \log\left(\frac{d_{\max}}{\epsilon} \sqrt{\frac{2n}{\alpha}}\right)\right). \quad (20)$$

When high precision is required and α is small, FWD-PUSH is likely not a local method, meaning per-epoch needs to touch the whole graph, hence the total complexity is about $O(m \log(1/\epsilon)/\alpha)$. Compared with the bound provided in Thm. 8, NAG method is $1/\sqrt{\alpha}$ times faster. However, whether there exists a local acceleration-based method like bound $O\left(\frac{\text{vol}(S)}{\sqrt{\alpha}} \log \frac{1}{\epsilon}\right)$ is still an open problem.

Polyak’s Heavy Ball (HB) method. Similar to NAG, another popular momentum-based acceleration method is the Heavy Ball method; different from NAG, the HB method updates $\mathbf{y}^t$ as:

$$\mathbf{y}^{t+1} = \mathbf{y}^t - \eta_t \nabla f(\mathbf{y}^t) + \beta_t (\mathbf{y}^t - \mathbf{y}^{t-1}),$$

where we set

$$\eta_t = \frac{4}{(\sqrt{2-\alpha} + \sqrt{\alpha})^2}, \quad \beta_t = \left(\frac{1-\kappa}{1+\kappa}\right)^2.$$

We can reach the following update method:

$$\mathbf{y}^{t+1} = 2(1-\alpha) \left[\frac{1+\kappa^2}{(1+\kappa)^2} \tilde{P} \mathbf{y}^t - \frac{(1-\kappa)^2}{(1+\kappa)^2} \mathbf{y}^{t-1} \right] + \frac{2\alpha(1+\kappa^2)}{(1+\kappa)^2} \tilde{\mathbf{s}}.$$

By changing variable $D^{1/2} \mathbf{y}^{t+1} = \mathbf{x}^{t+1}$, we have

$$\mathbf{x}^{t+1} = \frac{2\alpha(1+\kappa^2)}{(1+\kappa)^2} A D^{-1} \mathbf{x}^t - \frac{(1-\kappa)^2}{(1+\kappa)^2} \mathbf{x}^{t-1} + \frac{2(1-\alpha)(1+\kappa^2)}{(1+\kappa)^2} \mathbf{v}.$$

Compared with local linear methods such as CD and FWD-PUSH, the NAG and HB methods admit a better convergence rate, with total number of iterations about $O(m/\sqrt{\alpha})$, compared with linear ones $O(m/\alpha)$. Hence $O(1/\sqrt{\alpha})$ times faster. One can also consider the most popular method, the conjugate gradient method, which has a better convergence rate than standard gradient descent and is the same as our momentum-based methods. However, our two methods are easier to implement.

for any $\beta \in (0, 2)$. Let $\mu = \rho((I + D)^{-1}A)$. The optimal relaxation parameter is given by $\beta_{\text{opt}} = 1 + \left(\frac{\mu}{1 + \sqrt{1 - \mu^2}}\right)^2$.

In practice, explicitly computing μ is often infeasible for large-scale networks. We therefore adopt a simple heuristic strategy: initialize β close to 2 and gradually decrease it toward 1 until the number of required updates is minimized. Empirically, setting $\beta \approx 1.5$ yields a speedup of two to three times over the non-relaxed iteration on typical networks.

B.2.1 Quadratic optimization lens. We can rewrite the linear system of (1) into strongly convex optimization and then transform the problem into a strongly convex optimization problem. Recall our target linear system is $(I + L)z = s$. For an undirected graph, the matrix $I + D - A$ is symmetric normalized Laplacian. We define the minimization problem of a quadratic objective function f as the following

$$\arg \min_{z \in \mathbb{R}^n} \left\{ f(z) \triangleq \frac{1}{2} z^\top (I + D - A) z - \tilde{s}^\top z \right\}, \quad (18)$$

where $I + D - A$ is positive definite. Clearly, f is a strongly convex function, by taking the gradient $\nabla f(z)$ and letting it be zero, i.e. $\nabla f(z) := (I + D - A)z - \tilde{s} = 0$. To characterize the strongly convex and strong smoothness parameters of f (See definitions of strongly convex and smoothness in Appendix B), denote $\lambda_1, \lambda_2, \dots, \lambda_n$ as eigenvalues of L with $\lambda_1 = 1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_n \geq 0$. Therefore, by the fact from the graph spectral theory, given the normalized $I + D - A$, we have its eigenvalues $0 = \tilde{\lambda}_1 \leq \dots \leq \tilde{\lambda}_n \leq 2$. Therefore, the range of eigenvalues of $I - (1 - \alpha)\tilde{P}$ is $\lambda(I - (1 - \alpha)\tilde{P}) \in [\alpha, 2 - \alpha]$.

The above reformulation is commonly used in the optimization community and has also been studied in Fountoulakis et al.. It was observed that when the graph is undirected, FWD PUSH is a special case of a coordinate descent. The coordinate descent for the above problem (16) is

$$y^{t+1} = y^t - \eta_t \nabla_u f(y^t) \cdot e_u, \quad (19)$$

where each step size should be chosen such that $\eta_t \leq 1/L_u$, where L_u is the Lipschitz continuous parameter, $\|\nabla_u f(y + \delta e_u) - \nabla_u f(y)\| \leq L_u \cdot \delta$, for all $y \in \mathbb{R}^n$. Clearly, L_u corresponds to the diagonal of $I - (1 - \alpha)D^{-1/2}AD^{-1/2}$, which we always have $L_u \leq 1$. Replacing $\eta_t = 1$ and letting $-\nabla(y^t) = r^t$ in (17). We can recover FWD PUSH accordingly. The coordinate descent algorithm could be accelerated by choosing momentum strategy.

The challenge of obtaining faster iteration based on the accelerated coordinate descent method remains open due to the lack of linear-invariant property for the momentum method. Nevertheless, we use standard momentum techniques and leverage the advantage of continuous memory. Our momentum-based method can be expressed in a single line, and local linear convergence suggests that the number of iterations is

$$O\left(\frac{1}{\alpha \cdot \gamma_{1:T}} \log \frac{C_{\alpha,T}}{\epsilon}\right).$$

In the next section, we will demonstrate that one can save $1/\sqrt{\alpha}$ run time.

Now, if consensus is not reached, how should we quantify the cost of this lack of consensus? Here we observe that since the standard models use averaging as their basic mechanism, we can equivalently view nodes' actions in each time step as myopically optimizing a quadratic cost function:

Updating z_i as in Equation (1) is the same as choosing z_i to minimize

$$(z_i - s_i)^2 + \sum_{j \in N(i)} w_{i,j} (z_i - z_j)^2. \quad (2)$$

We therefore take this as the cost that i incurs by choosing a given value of z_i , so that averaging becomes a form of cost minimization.

Given this view, we can think of repeated averaging as the trajectory of best-response dynamics in a one-shot, complete information game played by the nodes in V , where i 's strategy is a choice of opinion z_i , and her payoff is the negative of the cost in Equation (2).

We first consider the case of undirected graphs and later handle the more general case of directed graphs. The main result in this section is a tight bound on the price of anarchy for the opinion-formation game in undirected graphs. After this, we discuss two slight extensions to the model: in the first, each player can put a different amount of weight on her internal opinion; and in the second, each player has several fixed opinions she listens to instead of an internal opinion. We show that both models can be reduced to the basic form of the model which we study first.

For undirected graphs we can simplify the social cost to the following form:

$$c(z) = \sum_i (z_i - s_i)^2 + 2 \sum_{(i,j) \in E, i > j} w_{i,j} (z_i - z_j)^2. \quad (20)$$

We can write this concisely in matrix form, by using the weighted Laplacian matrix L of G . L is defined by setting $L_{i,i} = \sum_{j \in N(i)} w_{i,j}$ and $L_{i,j} = -w_{i,j}$. We can thus write the social cost as

$$c(z) = z^T A z + \|z - s\|^2, \quad (21)$$

where the matrix $A = 2L$ captures the tension on the edges. The optimal solution is the y minimizing $c(\cdot)$. By taking derivatives, we see that the optimal solution satisfies

$$(A + I)y = s. \quad (22)$$

Since the Laplacian of a graph is a positive semidefinite matrix, it follows that $A + I$ is positive definite. Therefore, $(A + I)y = s$ has a unique solution:

$$y = (A + I)^{-1} s.$$

In the Nash equilibrium x each player chooses an opinion which minimizes her cost; in terms of the derivatives of the cost functions, this implies that $c'_i(x) = 0$ for all i . Thus, to find the players' opinions in the Nash equilibrium we should solve the following system of equations:

$$\forall i \quad (x_i - s_i) + \sum_{j \in N(i)} w_{i,j} (x_i - x_j) = 0.$$

Therefore in the Nash equilibrium each player holds an opinion which is a weighted average of her internal opinion and

the Nash equilibrium opinions of all her neighbors. This can be succinctly written as

$$(L + I)x = \left(\frac{1}{2}A + I\right)x = s. \quad (23)$$

As before, $\frac{1}{2}A + I$ is a positive definite matrix, and hence the unique Nash equilibrium is

$$x = \left(\frac{1}{2}A + I\right)^{-1} s.$$